

HabitPulse

Relazione di progetto

Alessandro Chiari

Indice

1	Introduzione.....	2
2	Obiettivo.....	2
3	Funzionalità principali.....	2
3.1	Accesso e profilo.....	2
3.2	Dashboard e attività.....	2
3.3	Progressi e analisi.....	2
4	Architettura e database.....	3
5	Tecnologie utilizzate.....	4
6	Sicurezza e stato del progetto.....	4
7	Verifiche effettuate.....	5
8	Manuale d'uso e installazione.....	6

1 Introduzione

HabitPulse è un'applicazione web full stack dedicata alla gestione delle abitudini quotidiane. Permette a ogni utente di registrare attività positive, chiamate habit, e comportamenti da limitare, chiamati vizi. Il progetto utilizza un'architettura client-server basata su React, Node.js, Express e MongoDB.

2 Obiettivo

L'obiettivo è offrire uno strumento semplice per:

- creare e organizzare le proprie attività quotidiane;
- registrare un contatore e un obiettivo giornaliero;
- distinguere le buone abitudini dai comportamenti da ridurre;
- osservare l'andamento nel tempo attraverso riepiloghi e grafici;
- mantenere separati i dati dei diversi utenti.

3 Funzionalità principali

3.1 Accesso e profilo

- **Registrazione:** creazione di un account con nome, cognome, email, password e telefono opzionale.
- **Login:** accesso tramite email e password; la password viene confrontata con l'hash salvato nel database.
- **Profilo:** modifica dei dati personali, cambio password, logout e cancellazione dell'account.

3.2 Dashboard e attività

- **Creazione:** inserimento di titolo, tipo, descrizione, date, colore, unità di misura e obiettivo.
- **Gestione:** modifica del colore e dell'obiettivo, riordinamento tramite drag and drop e arresto di un'attività.
- **Visualizzazione:** possibilità di mostrare o nascondere le attività terminate.

3.3 Progressi e analisi

- **Progresso giornaliero:** aggiornamento del contatore e del target della giornata.
- **Analisi:** filtri settimanali, mensili, annuali o personalizzati, grafici percentuali, copertura dei dati e calendario riepilogativo.

4 Architettura e database

Il frontend comunica con il backend tramite API REST che scambiano dati in formato JSON. Il server applica la logica dell'applicazione e usa Mongoose per leggere e scrivere i documenti su MongoDB Atlas.

Componente	Responsabilità
Client React	Interfaccia, navigazione, form, dashboard e grafici.
API Express	Rotte per autenticazione, habit, progressi e analisi.
MongoDB	Persistenza di utenti, attività e valori giornalieri.

4.1 Users

La collezione degli utenti contiene nome, cognome, email univoca, password cifrata, telefono e date di creazione e aggiornamento.

4.2 Habits

Ogni attività è collegata a un utente e contiene titolo, tipo (habit o vice), descrizione, data iniziale e finale, colore, ordine, obiettivo predefinito, unità di misura e stato di arresto.

4.3 DailyProgress

Il progresso giornaliero collega un'attività a una data e salva contatore e target. Un indice composto impedisce di avere due record per la stessa attività nello stesso giorno.

4.4 Principali endpoint

Area	Metodo	Endpoint
Autenticazione	POST	/api/auth/register - /login
Attività	GET/POST/PATCH	/api/habits
Progressi	GET/PATCH	/api/progress
Analisi	GET	/api/analysis

5 Tecnologie utilizzate

- **Frontend:** React 19, Vite, React Router DOM e Recharts.
- **Backend:** Node.js, Express.js, bcryptjs, CORS e dotenv.
- **Database:** MongoDB Atlas con ODM Mongoose.
- **Sviluppo:** ESLint, Nodemon e Concurrently.

6 Sicurezza e stato del progetto

Le password vengono cifrate con bcrypt prima del salvataggio. In fase di registrazione viene richiesta una password di almeno otto caratteri, con maiuscola, minuscola, numero e carattere speciale. L'email è unica nel database.

Il progetto è funzionante come prototipo locale, ma alcune parti devono essere completate prima di una pubblicazione reale:

- introdurre autenticazione e autorizzazione server-side tramite token o sessione;
- ricavare l'identità dell'utente dalla sessione invece di accettare liberamente userId dal client;
- spostare l'indirizzo delle API in una variabile di ambiente;
- limitare CORS alle origini autorizzate e aggiungere protezione dai tentativi ripetuti di login;
- eliminare in modo coerente habit e progressi quando viene cancellato un account;
- rimuovere e ruotare qualsiasi credenziale di database comparsa nel codice o nella cronologia Git;
- definire in modo esplicito il fuso orario usato per i progressi giornalieri.

6.1 Gestione dei progressi

Attualmente l'apertura della dashboard crea automaticamente il progresso del giorno con valore zero. Prima di considerare definitivo il calcolo della copertura, è necessario decidere se un giorno debba risultare tracciato solo dopo un'azione esplicita dell'utente.

7 Verifiche effettuate

Durante la revisione del progetto sono stati eseguiti controlli automatici sul frontend e sul backend. La tabella seguente riporta esclusivamente verifiche realmente eseguite sul codice attuale.

Controllo	Risultato	Nota
ESLint frontend	SUPERATO	Nessun errore segnalato.
Build Vite	SUPERATO	Build di produzione completata.
Sintassi Node.js	SUPERATO	Controller, modelli, rotte e configurazione validi.
Test automatici	ASSENTI	Non è presente una suite di test nel repository.

7.1 Test consigliati

- registrazione, login, modifica profilo e cambio password;
- isolamento dei dati tra due utenti diversi;
- creazione, modifica, arresto e riordinamento delle attività;
- aggiornamento concorrente dei progressi giornalieri;
- calcolo delle percentuali per habit e vizi;
- intervalli di date, cambio del giorno e fusi orari;
- cancellazione completa dell'account e dei dati collegati.

7.2 Nota sulle prestazioni

La build segnala un bundle JavaScript principale superiore a 500 kB. Il comportamento non impedisce l'avvio, ma in futuro è consigliabile caricare in modo differito la pagina di analisi e la libreria dei grafici.

8 Manuale d'uso e installazione

Per eseguire HabitPulse in locale sono necessari Node.js, npm e un database MongoDB raggiungibile.

8.1 Installazione

Dalla cartella principale del progetto installare le dipendenze dei tre livelli:

```
npm install
cd client && npm install
cd ../server && npm install
```

8.2 Configurazione del server

Creare un file .env nella cartella server senza inserirlo nel repository:

```
PORT=5000
MONGO_URI=mongodb+srv://<utente>:<password>@<cluster>/habitpulse
```

8.3 Avvio

Per avviare frontend e backend insieme, dalla cartella principale eseguire:

```
npm run dev
```

Il frontend è normalmente disponibile su <http://localhost:5173>, mentre il backend risponde su <http://localhost:5000>.

8.4 Utilizzo essenziale

1. Registrare un nuovo account oppure effettuare il login.
2. Creare una nuova attività dalla dashboard, scegliendo se si tratta di un habit o di un vizio.
3. Aggiornare ogni giorno contatore e obiettivo.
4. Usare il trascinamento per modificare l'ordine delle schede.
5. Consultare i riepiloghi dalla pagina Analysis e gestire dati, password e account dalla pagina Profile.